REINFORCEMENT LEARNING USING PYTHON

DATASETS

Datasets in reinforcement learning:

1. **CartPole**: This is a classic control problem where the goal is to balance a pole on a cart by applying forces to the cart. It's available in the OpenAI Gym library.
2. **MountainCar**: In this environment, the goal is to drive a car up a steep hill. The car must build up momentum to reach the top. This is also available in OpenAI Gym.
3. **LunarLander**: The objective is to land a lunar module safely on the moon's surface. This environment is slightly more complex but still beginner-friendly and available in OpenAI Gym.

DATASETS

1. **RLDS (Reinforcement Learning Datasets)**: This ecosystem includes tools to store, retrieve, and manipulate episodic data for reinforcement learning, learning from demonstrations, offline RL, or imitation learning
2. **D4RL (Datasets for Deep Data-Driven Reinforcement Learning)**: An open-source benchmark providing standardized environments and datasets for training and benchmarking RL algorithms
3. **RL Unplugged**: A suite of datasets including DMLab, Atari, Real World RL, Locomotion, and Control Suite datasets
4. **Robosuite**: Datasets generated with RLDS tools, focusing on robotic manipulation tasks
5. **Robomimic**: A suite of datasets for robotic imitation learning

PYTHON LIBRARIES USED FOR REINFORCEMENT LEARNING

1. **TensorFlow Agents (TF-Agents)**: An open-source library for building RL algorithms and environments using TensorFlow.
2. **OpenAI Gym**: A toolkit for developing and comparing reinforcement learning algorithms, providing a variety of environments.
3. **Stable Baselines3**: A set of reliable implementations of reinforcement learning algorithms.
4. **Ray RLlib**: A scalable library for reinforcement learning, supporting distributed computing.
5. **Keras-RL**: Integrates with Keras to provide easy-to-use RL algorithms.
6. **PyTorch RL**: Utilizes PyTorch for building and training RL models.
7. **Coach**: A library for training RL agents, developed by Intel AI Lab.

REINFORCEMENT LEARNING ON THE FROZEN LAKE ENVIRONMENT USING Q-LEARNING

```
pip install gym
```

#Import Libraries:

```
import gym
import numpy as np
```

Create the Environment:

```
env = gym.make('FrozenLake-v1')
```

Initialize Q-table:

```
q_table = np.zeros([env.observation_space.n, env.action_space.n])
```

We initialize the Q-table with zeros. The Q-table has dimensions [number of states, number of actions].

Set Hyperparameters:

```
alpha = 0.1  # Learning rate
gamma = 0.99  # Discount factor
epsilon = 0.1  # Exploration rate
```

alpha (learning rate): Determines how much new information overrides the old information.

gamma (discount factor): Determines the importance of future rewards.

epsilon (exploration rate): Controls the trade-off between exploration (choosing a random action) and exploitation (choosing the best-known action).

Training Parameters:

num_episodes = 10000

We set the number of episodes for training. Each episode is a complete run from the start state to a terminal state.

Training Loop:

```
for episode in range(num_episodes):
    state = env.reset()
    done = False
    while not done:
        if np.random.uniform(0, 1) < epsilon:
            action = env.action_space.sample()  # Explore action space
        else:
            action = np.argmax(q_table[state])  # Exploit learned values
        next_state, reward, done, _ = env.step(action)
        # Update Q-value using the Q-learning formula
        q_table[state, action] = q_table[state, action] + alpha * (reward + gamma * np.max(q_table[next_state])
- q_table[state, action])

        state = next_state
```

For each episode, we reset the environment to get the initial state.

We use an epsilon-greedy policy to choose an action: with probability epsilon, we explore by choosing a random action; otherwise, we exploit by choosing the action with the highest Q-value.

We take the chosen action and observe the next state and reward.

We update the Q-value

We update the current state to the next state and repeat until the episode is done.

Print the Trained Q-table:

```
print("Trained Q-table:")
print(q_table)
```

After training, we print the Q-table to see the learned values.

Test the Trained Agent:

```
state = env.reset()
done = False
steps = 0

while not done:
    action = np.argmax(q_table[state])
    state, reward, done, _ = env.step(action)
    steps += 1
    env.render()

print(f"Number of steps taken: {steps}")
```

We reset the environment and initialize the state.

We choose actions based on the trained Q-table (always exploiting the best-known action).

We render the environment to visualize the agent's actions.

We count the number of steps taken to reach the goal.